



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

Builder 模式讲义

—— 以“多层嵌套协议构造”为例（嵌入式语境）

1 Intent（意图）

将一个复杂对象的构建与其表示分离，使得同样的构建过程可以创建不同的表示。

在嵌入式协议场景中：

- 复杂对象 = 一帧多层嵌套协议报文
- 构建过程 = 按顺序拼装各层结构
- 表示 = 不同版本/不同安全级别/不同扩展字段的协议格式

核心理解：

把“如何构造报文”的算法独立出来，而不是让调用者手工拼 buffer。

2 Motivation（动机）

考虑一个嵌入式常见的多层协议帧结构：

```
Frame
├── HeadInfo
├── ExtHeader (optional)
├── SecurityBlock (optional)
├── Payload
│   ├── SubHeader
│   ├── TLV1
│   └── TLV2
└── CRC
```

在实际系统中，构造顺序通常是：

1. 先组装 Payload（TLV / 数据块）
2. 根据 Payload 长度生成 SubHeader
3. 如启用安全功能，进行加密或签名
4. 再填写 HeadInfo（总长度、版本、标志位等）
5. 最后计算 CRC

特点：

- 构造顺序与最终字节布局顺序不同
- HeadInfo 中的长度字段依赖 Payload
- SecurityBlock 依赖完整数据
- CRC 依赖整个 Frame
- 不同版本可能增加 ExtHeader 或 SecurityBlock

如果由调用者自行拼接：

- 容易遗漏长度回填
- 容易错误计算校验范围
- 容易破坏字段依赖关系
- 版本扩展会导致大量 if/else 分支

问题本质：

报文构造存在严格的步骤顺序和字段依赖关系，
同时协议结构会随着版本演进而变化。

3 Applicability（适用性）

在嵌入式多层协议中，当满足以下条件时适合使用 Builder：

✔ 对象复杂

- 多层嵌套
- 多字段依赖
- TLV 可变数量

✔ 构造顺序严格

- 必须先构造 Payload
- 再计算签名
- 再计算 CRC

✔ 同一构造流程支持不同表示

例如：

- 普通帧
- 加密帧
- 带扩展字段帧
- Debug 帧

构造步骤类似，但内部结构不同。

✔ 希望隔离表示细节

上层模块只知道：

“构造一个合法协议帧”

而不关心字段细节。

4 Structure（结构）

在协议场景中的角色映射：

GoF 角色	协议场景对应
Builder	报文构造接口
ConcreteBuilder	不同协议版本/安全级别实现
Director	报文构造流程控制
Product	最终字节流 buffer

结构逻辑：

```
Application
  ↓
Director (构建流程)
  ↓
Builder (抽象接口)
  ↓
ConcreteBuilder (具体协议格式)
  ↓
Buffer (最终报文)
```

5 Participants (参与者)

1. Builder

定义构建步骤接口，例如：

- build_header()
- build_security()
- build_payload()
- build_crc()

注意：

这里的关键不是函数名，而是：

步骤被标准化。

2. ConcreteBuilder

实现不同版本：

- Version1Builder
- SecureBuilder
- ExtendedBuilder

不同实现：

- 是否包含 Security Block
- TLV 顺序是否不同
- CRC 算法是否不同

3. Director

负责构建顺序，例如：

1. 构造 Payload
2. 构造 Header

3. 插入 Security
4. 计算 CRC
5. 输出 Frame

关键点：

Director 不关心具体字段结构。

4. Product

最终报文缓冲区。

它不暴露内部拼接细节。

6 Collaborations（协作）

构建过程：

1. Client 创建 ConcreteBuilder
2. Director 按顺序调用构建步骤
3. Builder 内部维护中间状态
4. 构建完成后返回完整 Frame

关键协作特征：

- Client 不直接操作 Buffer
 - Director 控制顺序
 - Builder 负责细节
-

7 Consequences（效果）

结合嵌入式协议场景分析。

① 分离构造算法与表示

构造算法固定：

- 先构造 payload
- 再插入 header
- 再计算校验

表示变化：

- 是否加密
- 是否签名
- TLV 顺序

好处：

协议版本升级时，不需要修改构造流程。

② 更好的结构控制

防止：

- CRC 计算范围错误
- 长度更新错误
- 可选字段遗漏

Builder 将不变量集中管理。

③ 易于扩展新协议版本

新增：

- 新安全机制
- 新扩展字段

只需新增 ConcreteBuilder。

④ 隔离复杂性

应用层代码变为：

“构造一帧合法报文”

而不是：

“逐字节拼接”

8 Implementation（实现要点）

在嵌入式 C 中实现时应注意：

① 内部维护中间状态

例如：

- 当前写入位置
- 当前 payload 长度
- 是否已构造安全块

Builder 必须维护这些。

② 防止非法构造顺序

可以：

- 在内部维护状态机
- 防止未完成 payload 就计算 CRC

③ 避免动态内存（嵌入式约束）

通常：

- 使用固定 buffer
- 预分配空间
- 构造完成后返回长度

④ Director 可以内嵌

在嵌入式中：

- Director 常为一个封装函数
- 不一定单独定义模块

9 与其他模式对比（协议场景）

模式	在协议构造中的角色
Factory	选择创建哪种 Builder
Strategy	选择 CRC 算法
Template Method	固定流程，但不隔离表示
Builder	隔离构造算法与表示

在多层嵌套协议中：

Builder 往往和 Strategy 组合使用。

10 总结（嵌入式视角）

在嵌入式多层嵌套协议中：

Builder 解决的问题不是：

“如何创建对象”

而是：

“如何保证复杂报文按正确顺序、正确依赖关系被构造”

当协议具有：

- 嵌套结构
- 版本差异
- 校验依赖
- 可选字段
- 安全模块

Builder 是一种合理且工程上可落地的组织方式。